KGiSL-IIM

co-Kreate your Genius

KGiSL INSTITUTE OF INFORMATION MANAGEMENT

Approved by *AICTE*, New Delhi and Affiliated to Bharathiar University

ISO 9001:2015 Certified Institution

KGiSL Campus, Saravanampatti, Coimbatore – 641 035

Focused Industry-ready Smart Talent

Master of Computer Applications

Year 20 - 20

Department of Computer Science

KGiSL INSTITUTE OF INFORMATION MANAGEMENT

COIMBATORE-641 035

KGiSL INSTITUTE OF INFORMATION MANAGEMENT
COIMBATORE - 641 035
(Affiliated to Bharathiar University)

DEPARTMENT OF COMPUTER SCIENCE

LABORATORY RECORD

NAME: _____ **ROLL No.:** _____

SEMESTER: _____ **YEAR:** _____

SUBJECT: _____

Certified Bonafide Record of Work Done by

Mr / Ms _____

Staff In Charge HOD

Place : _____

Date : _____

University Register No. _____

Submitted for the University Practical Examination held on _____

Internal Examiner External Examiner

INDEX

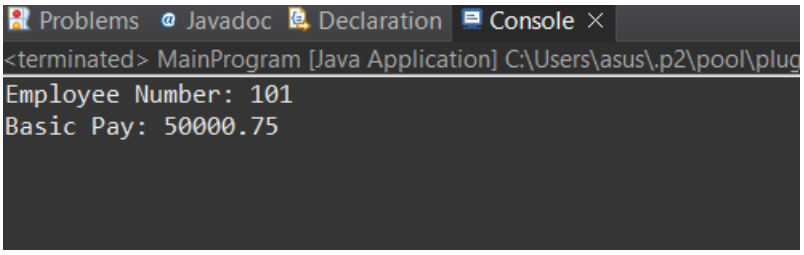
S.NO.	DATE	TITLE	PAGE NO	SIGNATURE
01		PACKAGE EMPLOYEE		
02		POLYMORPHISM, INHERITANCE AND INNER CLASS		
03		CUSTOM SHAPES		
04		MOUSE EVENT		
05		STUDENT INFORMATION		
06		TREE VIEWER		
07		PROGRAM TO PROHOBIT BAD WORDS		
08		TEMPERATURE CONVERTER		
09		PROGRAM TO MAINTAIN STUDENTS INFORMATION IN TEXT FILE		
10		IMAGE VIEWER		
11		2D SHAPES		
12		TRANSFORMATION		

Program 1: PACKAGE EMPLOYEE

```
package employee;
public class Employee {
   private int empNo;
   private double basicPay;
   public Employee(int empNo) {
      this.empNo = empNo;
   }
   public void setBasicPay(double pay) {
      this.basicPay = pay;
   }
   public void display() {
      System.out.println("Employee Number: " + empNo);
      System.out.println("Basic Pay: " + basicPay);
   }
}
```

```
package employee;
import employee.Employee;
public class MainProgram {
        public static void main(String[] args) {
                Employee emp1 = new Employee(101);
                emp1.setBasicPay(50000);
                emp1.setBasicPay(50000.75);
                emp1.display();
   }
}
```

OUTPUT:

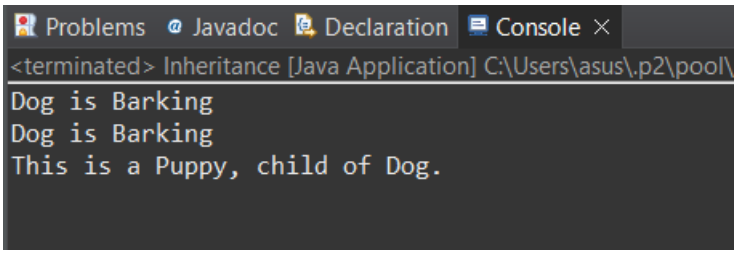

Problems
Javadoc
Declaration
Console
<terminated> MainProgram [Java Application] C:\Users\asus\.p2\pool\plug
Employee Number: 101
Basic Pay: 50000.75

Program 2: POLYMORPHRISM,INHERITANCE AND INNER CLASSES

```
package inheritance;
// Base class
class Animal {
    void sound() {
        System.out.println("Animal is making sound");
    }
}
// Subclass (Inheritance + Method Overriding)
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog is Barking");
    }
    // Inner class inside Dog
    class Puppy {
        void display() {
            System.out.println("This is a Puppy, child of Dog.");
        }
    }
}
// Main class
public class Inheritance {
    public static void main(String[] args) {
        // Polymorphism: Parent reference -> Child object
        Animal myAnimal = new Dog();
        myAnimal.sound(); // Output: Dog is Barking
        // Inheritance (Overriding method)
        Dog myDog = new Dog();
        myDog.sound(); // Output: Dog is Barking
```

```
        // Inner class usage
        Dog.Puppy myPuppy = myDog.new Puppy();
        myPuppy.display(); // Output: This is a Puppy, child of Dog.
    }
}
```

OUTPUT:



Program 3:CUSTOM SHAPES

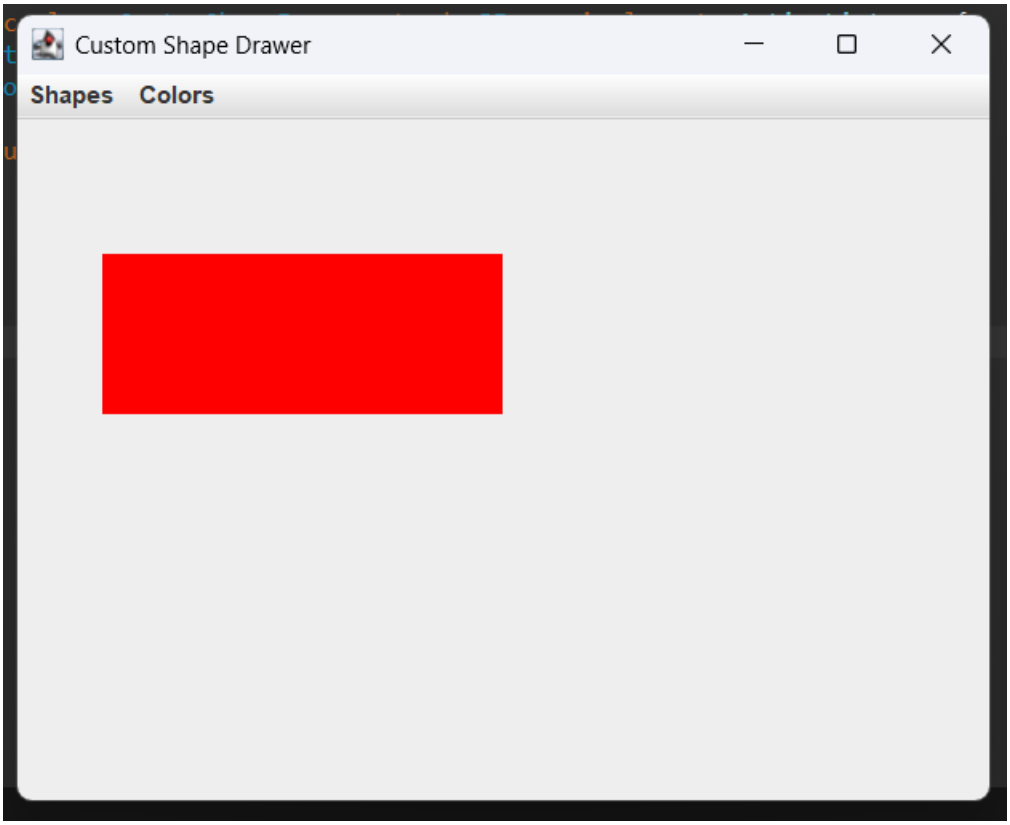
```
package java_lab;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class CustomShapeFrame extends JFrame implements ActionListener {
   String selectedShape = "None";
   Color selectedColor = Color.BLACK;
   public CustomShapeFrame(int width, int height, int x, int y) {
      setTitle("Custom Shape Drawer");
      setSize(width, height);
      setLocation(x, y);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      // Create menu bar
      JMenuBar menuBar = new JMenuBar();
      // Shape menu
      JMenu shapeMenu = new JMenu("Shapes");
      String[] shapes = {"Line", "Rectangle", "Oval"};
      for (String shape : shapes) {
         JMenuItem item = new JMenuItem(shape);
         item.addActionListener(this);
         shapeMenu.add(item);
      }
      // Color menu
      JMenu colorMenu = new JMenu("Colors");
      JMenuItem red = new JMenuItem("Red");
      JMenuItem blue = new JMenuItem("Blue");
      JMenuItem green = new JMenuItem("Green");
      red.addActionListener(this);
      blue.addActionListener(this);
```

```
    green.addActionListener(this);
    colorMenu.add(red);
    colorMenu.add(blue);
    colorMenu.add(green);
    menuBar.add(shapeMenu);
    menuBar.add(colorMenu);
    setJMenuBar(menuBar);
}
@Override
public void paint(Graphics g) {
    super.paint(g);
    g.setColor(selectedColor);
    switch (selectedShape) {
        case "Line":
            g.drawLine(50, 100, 250, 100);
            break;
        case "Rectangle":
            g.fillRect(50, 120, 200, 80);
            break;
        case "Oval":
            g.fillOval(50, 120, 200, 80);
            break;
    }
}
@Override
public void actionPerformed(ActionEvent e) {
    String cmd = e.getActionCommand();
    if (cmd.equals("Line") || cmd.equals("Rectangle") || cmd.equals("Oval")) {
        selectedShape = cmd;
    } else if (cmd.equals("Red")) {
        selectedColor = Color.RED;
```

```
        } else if (cmd.equals("Blue")) {
            selectedColor = Color.BLUE;
        } else if (cmd.equals("Green")) {
            selectedColor = Color.GREEN;
        }
        repaint();
    }
    public static void main(String[] args) {
        CustomShapeFrame frame = new CustomShapeFrame(500, 400, 200, 200);
        frame.setVisible(true);
    }
}
```

OUTPUT:



Program 4: MOUSE EVENT

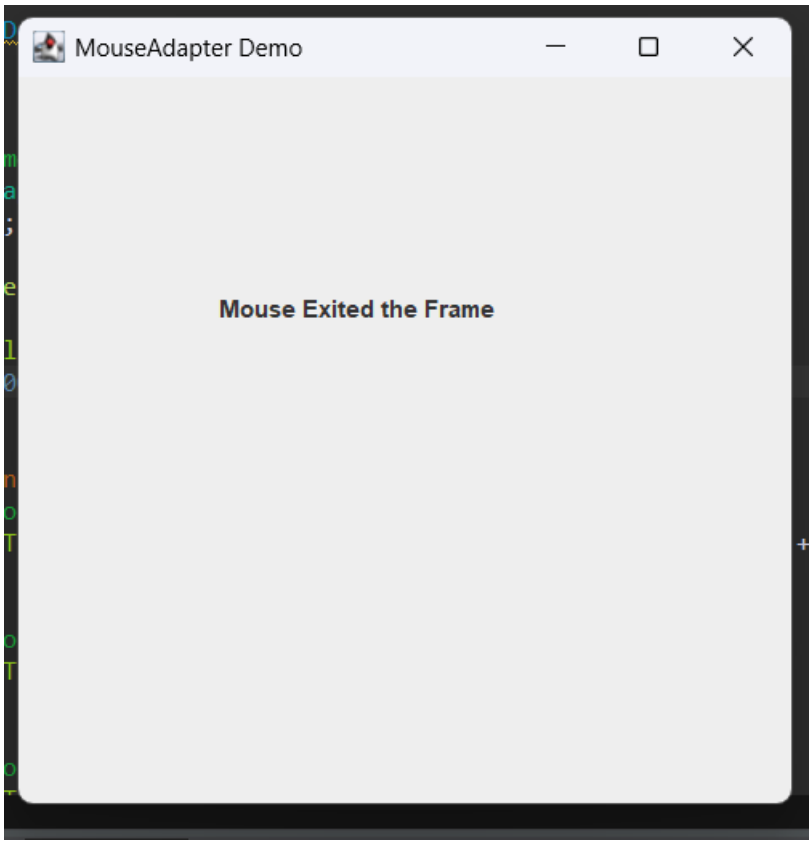
```
package java_lab;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MouseAdapterDemo extends JFrame {
   JLabel label;
   public MouseAdapterDemo() {
      setTitle("MouseAdapter Demo");
      setSize(400, 400);
      setLayout(null);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      label = new JLabel("Move or click the mouse");
      label.setBounds(100, 100, 250, 30);
      add(label);
      addMouseListener(new MouseAdapter() {
         public void mouseClicked(MouseEvent e) {
            label.setText("Mouse Clicked at (" + e.getX() + ", " + e.getY() + ")");
         }
         public void mousePressed(MouseEvent e) {
            label.setText("Mouse Pressed");
         }
         public void mouseReleased(MouseEvent e) {
            label.setText("Mouse Released");
         }
         public void mouseEntered(MouseEvent e) {
            label.setText("Mouse Entered the Frame");
         }
         public void mouseExited(MouseEvent e) {
            label.setText("Mouse Exited the Frame");
```

```
        }
      });
    }

    public static void main(String[] args) {
      MouseAdapterDemo frame = new MouseAdapterDemo();
      frame.setVisible(true);
    }
}
```

OUTPUT:



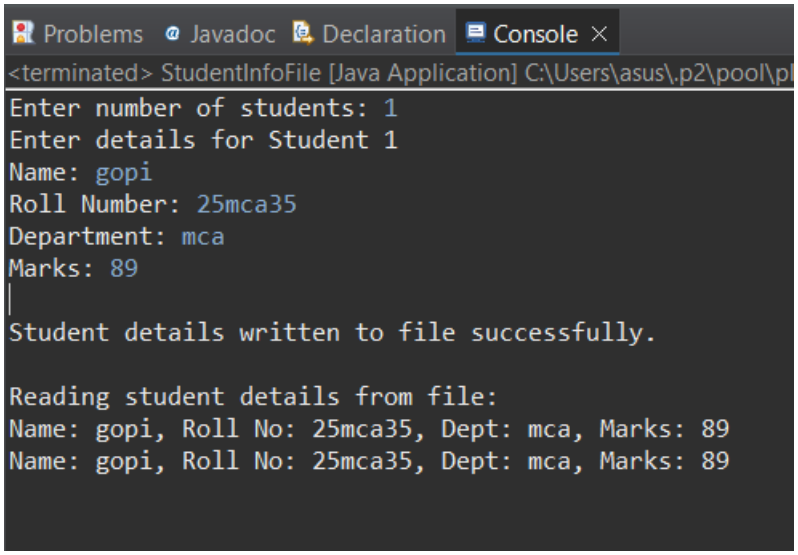
Program 5: STUDENT INFO

```
package java_lab;
import java.io.*;
import java.util.Scanner;

public class StudentInfoFile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String fileName = "students.txt";
        try {
            // Writing student info
            FileWriter fw = new FileWriter(fileName, true);
            BufferedWriter bw = new BufferedWriter(fw);
            System.out.print("Enter number of students: ");
            int n = sc.nextInt();
            sc.nextLine(); // consume newline
            for (int i = 0; i < n; i++) {
                System.out.println("Enter details for Student " + (i + 1));
                System.out.print("Name: ");
                String name = sc.nextLine();
                System.out.print("Roll Number: ");
                String roll = sc.nextLine();
                System.out.print("Department: ");
                String dept = sc.nextLine();
                System.out.print("Marks: ");
                int marks = sc.nextInt();
                sc.nextLine(); // consume newline
                bw.write(name + "," + roll + "," + dept + "," + marks);
                bw.newLine();
            }
            bw.close();
```

```
            fw.close();
            System.out.println("\nStudent details written to file successfully.");
            // Reading student info
            System.out.println("\nReading student details from file:");
            BufferedReader br = new BufferedReader(new FileReader(fileName));
            String line;
            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                System.out.println("Name: " + parts[0] + ", Roll No: " + parts[1] + ", Dept: " + parts[2] + ", Marks: " + parts[3]);
            }
            br.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:



Program 6: TREE VIEWER

```
package java_lab;


import javax.swing.*;
import javax.swing.tree.DefaultMutableTreeNode;
import java.awt.*;
public class TreeViewer {
   public static void main(String[] args) {
      // Create a JFrame
      JFrame frame = new JFrame("Tree Viewer Example");
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      frame.setSize(400, 300);

      // Create the root node
      DefaultMutableTreeNode root = new DefaultMutableTreeNode("Root");

      // Create child nodes
      DefaultMutableTreeNode branch1 = new DefaultMutableTreeNode("Branch 1");
      DefaultMutableTreeNode branch2 = new DefaultMutableTreeNode("Branch 2");

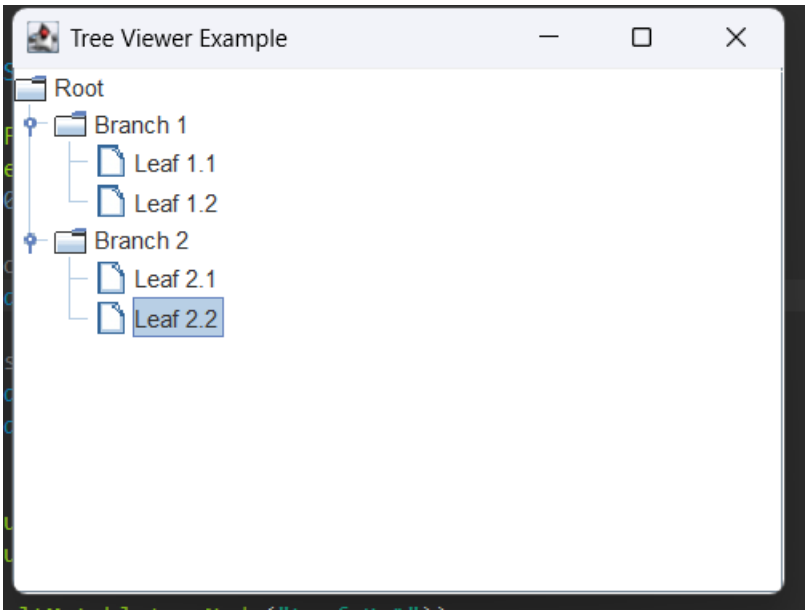
      // Add leaf nodes to branches
      branch1.add(new DefaultMutableTreeNode("Leaf 1.1"));
      branch1.add(new DefaultMutableTreeNode("Leaf 1.2"));

      branch2.add(new DefaultMutableTreeNode("Leaf 2.1"));
      branch2.add(new DefaultMutableTreeNode("Leaf 2.2"));

      // Add branches to root
      root.add(branch1);
      root.add(branch2);
```

```
        // Create the JTree using the root node
        JTree tree = new JTree(root);
        // Put the tree in a scroll pane
        JScrollPane scrollPane = new JScrollPane(tree);
        // Add the scroll pane to the frame
        frame.add(scrollPane, BorderLayout.CENTER);
        // Make the frame visible
        frame.setVisible(true);
    }
}
```

OUTPUT:



Program 7: BADWORDS

```
package java_lab;
import java.io.*;
import java.util.*;

public class BadWordFilter {
   private static final Set<String> BAD_WORDS = new HashSet<>(Arrays.asList(
      "badword1", "badword2", "hate", "violence"
   ));

   public static void main(String[] args) {
      String filePath = "C:\\Users\\asus\\Documents\\sample.txt"; // Change path as needed

      try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
         String line;
         boolean hasBadWords = false;

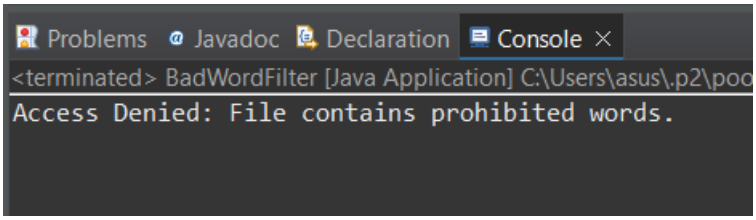
         while ((line = reader.readLine()) != null) {
            for (String word : BAD_WORDS) {
               if (line.toLowerCase().contains(word.toLowerCase())) {
                  hasBadWords = true;
                  break;
               }
            }
            if (hasBadWords) break;
         }

         if (hasBadWords) {
            System.out.println("Access Denied: File contains prohibited words.");
         } else {
            try (BufferedReader displayReader = new BufferedReader(new FileReader(filePath))) {
```

```
                System.out.println("File Contents:");
                while ((line = displayReader.readLine()) != null) {
                    System.out.println(line);
                }
            }
        }

    } catch (FileNotFoundException e) {
        System.out.println("File not found: " + filePath);
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }
  }
}
```

OUTPUT:



Program 8: TEMPERATURE_CONVERTER

```
package java_lab;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class TemperatureConverter extends JFrame implements ActionListener {
    private JTextField celsiusField, fahrenheitField;
    private JButton convertToF, convertToC, clearButton;

    public TemperatureConverter() {
        setTitle("Temperature Converter");
        setSize(350, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridLayout(4, 2, 10, 10));

        JLabel celsiusLabel = new JLabel("Celsius:");
        JLabel fahrenheitLabel = new JLabel("Fahrenheit:");

        celsiusField = new JTextField();
        fahrenheitField = new JTextField();

        convertToF = new JButton("Convert to Fahrenheit");
        convertToC = new JButton("Convert to Celsius");
        clearButton = new JButton("Clear");

        convertToF.addActionListener(this);
        convertToC.addActionListener(this);
        clearButton.addActionListener(this);
```

```
    add(celsiusLabel);
    add(celsiusField);
    add(fahrenheitLabel);
    add(fahrenheitField);
    add(convertToF);
    add(convertToC);
    add(clearButton);

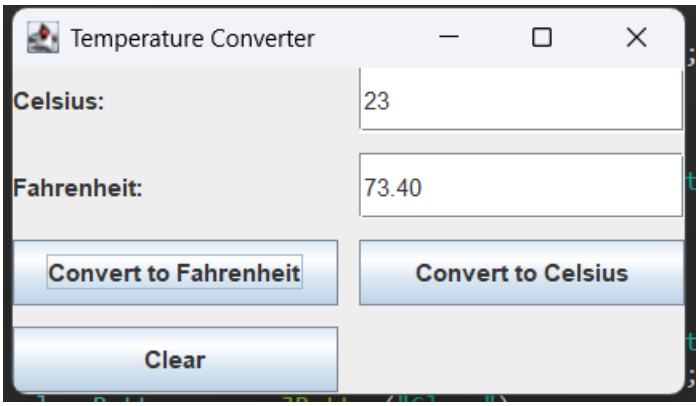
    setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    try {
        if (e.getSource() == convertToF) {
            String celsiusText = celsiusField.getText();
            if (!celsiusText.isEmpty()) {
                double celsius = Double.parseDouble(celsiusText);
                double fahrenheit = (celsius * 9 / 5) + 32;
                fahrenheitField.setText(String.format("%.2f", fahrenheit));
            }
        } else if (e.getSource() == convertToC) {
            String fahrenheitText = fahrenheitField.getText();
            if (!fahrenheitText.isEmpty()) {
                double fahrenheit = Double.parseDouble(fahrenheitText);
                double celsius = (fahrenheit - 32) * 5 / 9;
                celsiusField.setText(String.format("%.2f", celsius));
            }
        } else if (e.getSource() == clearButton) {
            celsiusField.setText("");
            fahrenheitField.setText("");
        }
```

```
            } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Please enter a valid number.", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new TemperatureConverter());
    }
}
```

OUTPUT:



Program 9:STUDENT_FX

```
package java_lab;

import javax.swing.*;
import java.awt.*;

public class StudentDetailsSwing extends JFrame {

    public StudentDetailsSwing() {
        setTitle("Student Details");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridLayout(6, 2, 10, 10));

        JLabel nameLabel = new JLabel("Name:");
        JTextField nameField = new JTextField();

        JLabel regLabel = new JLabel("Register No:");
        JTextField regField = new JTextField();

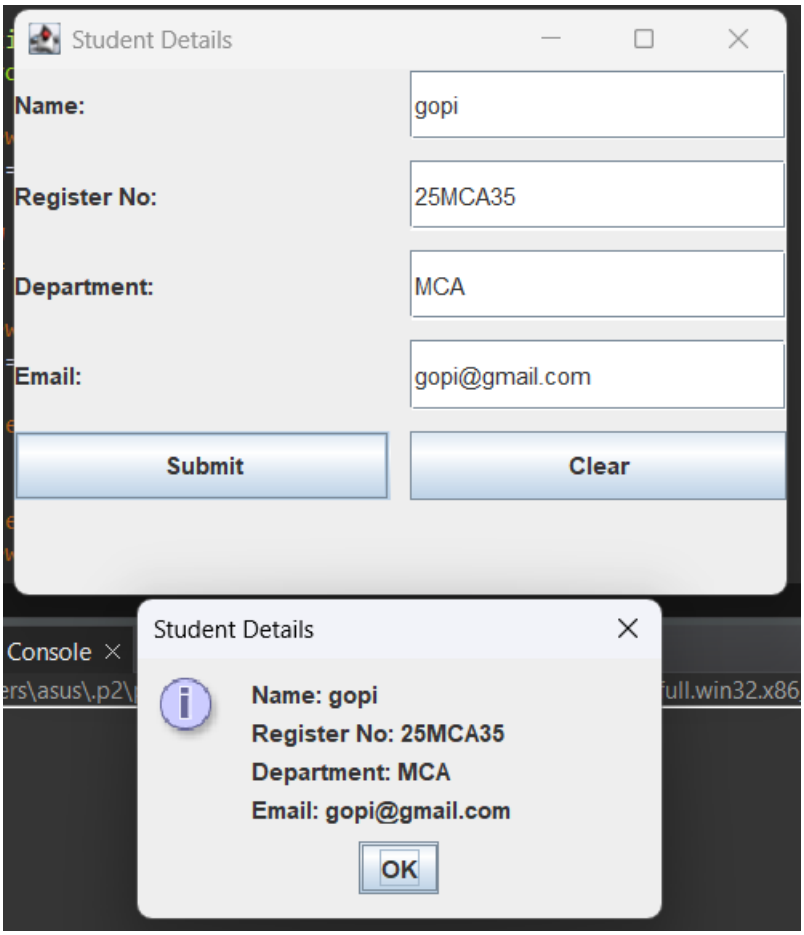
        JLabel deptLabel = new JLabel("Department:");
        JTextField deptField = new JTextField();

        JLabel emailLabel = new JLabel("Email:");
        JTextField emailField = new JTextField();
        JButton submitBtn = new JButton("Submit");
        JButton clearBtn = new JButton("Clear");
        add(nameLabel); add(nameField);
        add(regLabel); add(regField);
        add(deptLabel); add(deptField);
```

```
        add(emailLabel); add(emailField);
        add(submitBtn); add(clearBtn);
        // Action for Submit
        submitBtn.addActionListener(e -> {
            String name = nameField.getText();
            String reg = regField.getText();
            String dept = deptField.getText();
            String email = emailField.getText();
            if (name.isEmpty() || reg.isEmpty() || dept.isEmpty() || email.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Please fill all fields.");
            } else {
                String info = "Name: " + name + "\nRegister No: " + reg +
                        "\nDepartment: " + dept + "\nEmail: " + email;
                JOptionPane.showMessageDialog(this, info, "Student Details",
JOptionPane.INFORMATION_MESSAGE);
            }
        });
        // Action for Clear
        clearBtn.addActionListener(e -> {
            nameField.setText("");
            regField.setText("");
            deptField.setText("");
            emailField.setText("");
        });
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(StudentDetailsSwing::new);
    }
}
```

OUTPUT:


Student Details
Name:
gopi
Register No:
25MCA35
Department:
MCA
Email:
gopi@gmail.com
Submit
Clear
Console
Student Details
Name: gopi
Register No: 25MCA35
Department: MCA
Email: gopi@gmail.com
OK

Program 10: IMAGE_VIEWER

```
package image;

import javax.swing.*;
import java.awt.*;
import java.io.File;
import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.beans.*;

public class ImageViewerBean extends JPanel implements java.io.Serializable {
   private BufferedImage image;

   public ImageViewerBean() {
      // Default constructor
   }

   public void setImagePath(String path) {
      try {
         image = ImageIO.read(new File(path));
         repaint();
      } catch (Exception e) {
         System.out.println("Image loading failed: " + e.getMessage());
      }
   }

   public String getImagePath() {
      return "";
   }

   protected void paintComponent(Graphics g) {
```

```
        super.paintComponent(g);
        if (image != null) {
            int x = (getWidth() - image.getWidth()) / 2;
            int y = (getHeight() - image.getHeight()) / 2;
            g.drawImage(image, x, y, this);
        }
    }
}
package image;

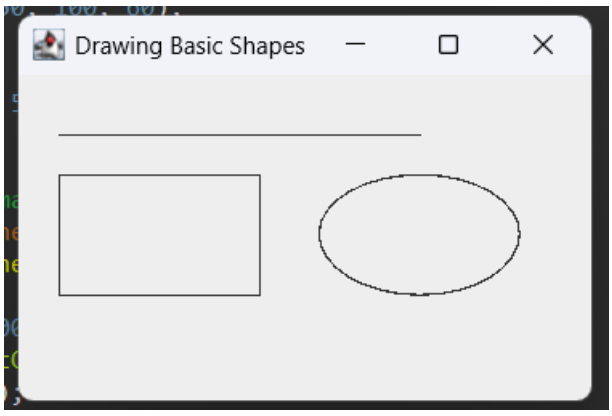
import javax.swing.*;

public class ImageViewertest {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Java Bean Image Viewer");
        ImageViewerBean viewer = new ImageViewerBean();

        viewer.setImagePath("C:/Users/Admin/image1.png"); // 🔁 Change to your image path

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(600, 400);
        frame.add(viewer);
        frame.setVisible(true);
    }
}
```

OUTPUT:



Program 11: 2D_SHAPES

```
package java_lab;

import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Graphics;

public class ShapeDrawing extends JPanel {

    public void paintComponent(Graphics g) {
        super.paintComponent(g);

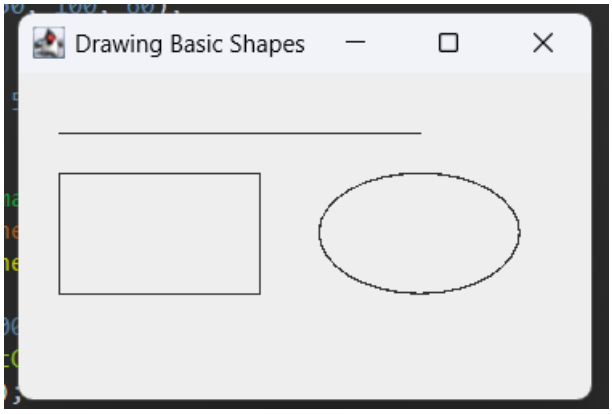
        // Draw a line
        g.drawLine(20, 30, 200, 30);

        // Draw a rectangle
        g.drawRect(20, 50, 100, 60);

        // Draw an oval
        g.drawOval(150, 50, 100, 60);
    }

    public static void main(String[] args) {
        JFrame frame = new JFrame("Drawing Basic Shapes");
        ShapeDrawing panel = new ShapeDrawing();
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.add(panel);
        frame.setVisible(true);
    }
}
```

OUTPUT:



Program 12:TRANSFORMATIONS

```
package java_lab;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class TransformShapes extends JPanel implements ActionListener {
   private int choice = 0;  // 1=translate, 2=scale, 3=rotate

   public TransformShapes() {
      JFrame frame = new JFrame("2D Transformations - Line & Rectangle");
      frame.setSize(500, 400);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

      // Buttons for transformations
      JButton translateBtn = new JButton("Translate");
      JButton scaleBtn = new JButton("Scale");
      JButton rotateBtn = new JButton("Rotate");

      translateBtn.addActionListener(this);
      scaleBtn.addActionListener(this);
      rotateBtn.addActionListener(this);

      JPanel buttonPanel = new JPanel();
      buttonPanel.add(translateBtn);
      buttonPanel.add(scaleBtn);
      buttonPanel.add(rotateBtn);

      frame.add(this, BorderLayout.CENTER);
      frame.add(buttonPanel, BorderLayout.SOUTH);
```

```
    frame.setVisible(true);
}

@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;

    // Original shape
    g2.setColor(Color.BLUE);
    g2.drawRect(100, 100, 100, 50); // rectangle
    g2.drawLine(100, 200, 200, 250); // line
    g2.drawString("Original", 90, 90);

    // Apply transformations
    g2.setColor(Color.RED);
    switch (choice) {
        case 1: // Translation
            g2.translate(100, 50);
            g2.drawRect(100, 100, 100, 50);
            g2.drawLine(100, 200, 200, 250);
            g2.drawString("Translated", 90, 90);
            break;

        case 2: // Scaling
            g2.translate(100, 50);
            g2.scale(1.5, 1.5);
            g2.drawRect(100, 100, 100, 50);
            g2.drawLine(100, 200, 200, 250);
            g2.drawString("Scaled", 90, 90);
```

```
            break;

        case 3: // Rotation
            g2.translate(250, 150);
            g2.rotate(Math.toRadians(45));
            g2.drawRect(0, 0, 100, 50);
            g2.drawLine(0, 100, 100, 150);
            g2.drawString("Rotated", 10, -10);
            break;
    }
  }

  @Override
  public void actionPerformed(ActionEvent e) {
    String cmd = e.getActionCommand();
    if (cmd.equals("Translate")) choice = 1;
    else if (cmd.equals("Scale")) choice = 2;
    else if (cmd.equals("Rotate")) choice = 3;
    repaint();
  }

  public static void main(String[] args) {
    SwingUtilities.invokeLater(TransformShapes::new);
  }
}
```

OUTPUT:

